

Manufacturing Quality Intelligence System (MQIS) A Real-Time Defect Detection and RAG-Based Quality Reporting Platform

Bharat Bhardwaj*

Rishit†

Asmita‡

August 16, 2026

Project Proposal for UCS503

Submitted to: Asif Jeelani

Thapar Institute of Engineering and Technology

1 Higher-order Goal

This project aims to improve manufacturing quality assurance by reducing the time and cognitive effort required to detect, log, and diagnose defects on a production line. While automated visual inspection already exists in industry, the immediate engineering objective of this project is to build a deployable, end-to-end software pipeline that not only flags defective parts but also *explains* why they are occurring, using live analytics combined with retrieval-augmented generation (RAG).

2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core detection-to-logging pipeline first, and validating it against a public defect dataset before adding intelligence layers.
- We will prioritize fast feedback over perfection by using weekly milestones, testing each module (detection, database, analytics, RAG) independently before integration.
- Success is defined by what can be reliably detected, logged, and reported in the first working prototype, then iteratively improved with better classification and reporting quality.

3 Problem Statement

Manufacturing lines in small and mid-sized facilities frequently rely on manual visual inspection to catch defective parts. This approach faces several concrete issues:

- **Inconsistency:** Human inspectors tire and miss defects, especially over long shifts.
- **No root-cause visibility:** A rejected part is simply discarded; nothing connects the rejection to *which* line, *which* time window, or *which* recurring failure pattern is responsible.

*Roll No: 1024240013, Email: bbhardwaj_be24@thapar.edu

†Roll No: 1024240030, Email: rgoel1_be24@thapar.edu

‡Roll No: 1024240014, Email: aasmita_be24@thapar.edu

- **Delayed reporting:** Shift reports summarizing what failed and why are compiled manually, well after the fact, delaying corrective action.
- **Siloed knowledge:** Experienced technicians know that certain defect signatures map to specific causes (e.g., a scratch pattern indicating a worn clamp), but this knowledge is not captured anywhere the system can use it automatically.

Why this matters now: As production lines scale in number and throughput, even a small delay in detecting a systemic defect (e.g., a mis-calibrated press) can result in a large batch of rejected output before a human notices the pattern.

4 Proposed Solution — Software Proposition

4.1 Overview

We propose **MQIS**, a web-based defect detection and quality-intelligence dashboard with four integrated components:

- **Vision-based inspection module:** captures/loads a part image and compares it against a known-good reference using image differencing and contour analysis to localize and classify the defect (type, severity, confidence).
- **Event logging layer:** every inspection — pass or reject — is persisted with a timestamp, production line ID, defect type, severity, and confidence score.
- **Live analytics engine:** aggregates the log in real time into per-line rejection rates, line status flags (Normal / Warning / Critical), defect-type breakdowns, and hourly trend data (including the peak-failure hour).
- **RAG-based reporting assistant:** embeds a small factory knowledge base (defect causes, maintenance notes) into a vector store, retrieves the most relevant knowledge for the current data snapshot, and uses a local LLM to generate a grounded, human-readable shift report or answer free-text operator questions.

4.2 Core Workflow

1. An inspector (or the simulated feed) triggers an inspection for a selected production line.
2. The vision module compares the part against a reference image, localizes any defect region, and classifies it by type and severity.
3. The result is logged to the database and reflected immediately on the live dashboard.
4. The analytics engine recomputes rejection rates, line status, and defect distributions.
5. On request, the RAG assistant retrieves relevant factory knowledge and produces a shift report or answers a specific question (e.g., “why is Line 1 rejecting more today?”).

4.3 Operational Constraints

- The dashboard must remain responsive on standard laptop-class hardware (no dedicated GPU required for inspection; the LLM component runs locally via a lightweight quantized model).
- Avoid dependence on proprietary cloud vision APIs; use open, inspectable computer-vision techniques so the system’s decisions remain explainable.

- Design the schema and analytics layer to be dataset-agnostic, so the same pipeline can later be pointed at a different defect category or product line with minimal changes.

5 Solution Approach — Engineering Focus

This is an engineering systems project; the focus is on building a correct, integrated pipeline rather than on inventing new defect-detection algorithms.

5.1 Detection Module (Non-research)

- Reference-based differencing: grayscale conversion, Gaussian blur, absolute difference against a known-good image.
- Thresholding and morphological operations (close/open) to isolate the defect region while suppressing noise.
- Contour extraction to obtain a bounding box for visualization, with a minimum-area filter to reject spurious detections.
- Defect classification is data-driven (based on labelled categories in the dataset) rather than a novel model, keeping scope realistic for the semester.

5.2 System Architecture (3-tier)

- **Frontend:** Streamlit-based dashboard — inspection view, live statistics, defect log table, and an AI assistant panel.
- **Backend / logic:** Python modules for detection, analytics aggregation, and RAG orchestration.
- **Data layer:** SQLite for structured event logging; a local vector store (Chroma) for the factory knowledge base used in retrieval.

5.3 CI/CD and Iteration Discipline

- Each module (detector, database, analytics, RAG) is developed and unit-tested independently before integration into `app.py`.
- A shared Git repository with a linear commit history per feature branch, merged weekly.
- Lightweight automated checks (import sanity, schema validation) run before each merge to avoid breaking the dashboard.

6 Evaluation Criteria

6.1 Primary Evaluation Metric

Mean Time-to-Log (MTTL): the time between an inspection event occurring and it being reflected in the live dashboard and database.

- Target: $MTTL \leq 2$ seconds per inspection under normal load.
- Attribution: measured via internal timestamps between capture and successful database write.

6.2 Secondary Evaluation Metrics

- **Detection consistency:** percentage of known defective samples correctly flagged as rejected across the test dataset.
- **Report generation latency:** time taken by the RAG pipeline to produce a full shift report (target: under 30 seconds on local hardware).
- **Analytics correctness:** rejection-rate and status-flag calculations validated against manually computed values on a fixed sample log.
- **Usability:** informal feedback (1–5 scale) from 3–5 peer testers on dashboard clarity and report usefulness.

6.3 Validation Plan

Run a fixed batch of ≥ 50 inspections across three simulated production lines, compare system-reported rejection rates and status flags against a manually tallied ground truth, and collect qualitative feedback on the generated shift reports from 3–5 peer reviewers.

7 Scalability

1. **Theoretical:** The architecture decouples the detection, logging, analytics, and reporting layers, allowing each to scale independently (e.g., swapping SQLite for a managed relational database under higher throughput) without redesigning the pipeline.
2. **Practical:** The knowledge base and vector store can be extended with additional factory documentation without code changes, and the detection module can be pointed at a new defect category by supplying a new reference image set and label mapping.

8 Engine Availability Heuristic

A mature set of tools is available to implement this without building infrastructure from scratch:

- **OpenCV** — image differencing, thresholding, contour detection.
- **Streamlit** — rapid dashboard/UI development.
- **Pandas** — analytics aggregation (rejection rates, groupby summaries).
- **SQLite** — lightweight structured event storage.
- **ChromaDB + Sentence-Transformers** — embedding and retrieval for the knowledge base.
- **Ollama (Llama 3.2)** — local LLM inference for report generation, avoiding dependence on external API costs or connectivity.

We will use these mature, well-documented components to focus effort on system integration, correctness, and evaluation rather than reinventing infrastructure.

9 Project Scope and Deliverables

9.1 Initial Deliverable (Milestone 1)

- Working detection module against a labelled reference dataset.
- SQLite logging of every inspection event.
- Minimal Streamlit dashboard showing the latest inspection result.

9.2 Subsequent Deliverables

- Full live-analytics panel (per-line status, defect-type/hourly breakdowns).
- RAG knowledge base integration with shift-report generation.
- Free-text question-answering assistant grounded in live data.
- Final testing, documentation, and demo packaging.

10 Team Roles and Resources

- **Bharat Bhardwaj** — Vision/detection module, system integration across all three components, dataset preparation, and evaluation of detection consistency.
- **Rishit** — Database schema, analytics engine, dashboard integration (Streamlit frontend).
- **Asmita** — RAG pipeline (knowledge base, embeddings, LLM integration), documentation, and final report writing.

Material resources: A public defect-detection image dataset (e.g., MVTec-style categories), standard laptop hardware, and open-source software only. No external funding is required; all tools used are free and open-source.

11 15-Week Timeline

Week	Bharat (Detection & Integration)	Rishit (Data/Analytics)	Asmita (RAG/Reporting)
1	Literature/tooling survey; finalize dataset	Define DB schema for event logging	Survey RAG stack (Chroma, embeddings, Ollama)
2	Set up reference-image comparison pipeline	Implement <code>database.py</code> (init, log, query)	Set up local Llama 3.2 via Ollama; test basic chat call
3	Implement grayscale diff + thresholding	Test DB read/write with dummy data	Draft initial factory knowledge base (.txt notes)
4	Add contour extraction + bounding box	Build minimal Streamlit shell (<code>app.py</code> skeleton)	Implement embedding pipeline for knowledge base

Week	Bharat (Detection & Integration)	Rishit (Data/Analytics)	Asmita (RAG/Reporting)
5	Implement defect classification mapping	Wire detection output into DB logging	Implement Chroma collection build/query
6	Milestone 1: Working detection on sample dataset	Milestone 1: Logging pipeline integrated	Milestone 1: Retrieval returns relevant chunks
7	Tune thresholds; reduce false positives	Build <code>analytics.py</code> : rejection rate, by-line summary	Draft prompt template for shift report generation
8	Add severity/confidence scoring	Add by-type and by-hour breakdowns	Implement <code>generate_report()</code> end-to-end
9	Visual annotation (bounding box + label overlay)	Add line-status flags (Normal/Warning/Critical)	Implement <code>answer_question()</code> for free-text queries
10	Integration testing with live dashboard	Live stats panel in Streamlit (metrics, bar chart)	Integrate AI assistant panel into dashboard UI
11	Milestone 2: Full inspection view working	Milestone 2: Full analytics panel working	Milestone 2: Full RAG panel working
12	End-to-end integration (all modules combined)	End-to-end integration (all modules combined)	End-to-end integration (all modules combined)
13	System testing: run ≥ 50 inspections, log accuracy	Validate analytics against manual ground truth	Evaluate report quality/latency against target
14	Bug fixing; buffer week for unresolved issues	Bug fixing; buffer week for unresolved issues	Bug fixing; buffer week for unresolved issues
15	Final demo prep, documentation, presentation	Final demo prep, documentation, presentation	Final demo prep, documentation, presentation

12 Risks and Mitigations

- **Detection accuracy on edge cases:** lighting/angle variation may cause false positives; mitigate with a fixed capture setup and threshold tuning during Weeks 6–8.
- **LLM latency/resource constraints:** local LLM inference may be slow on limited hardware; mitigate by using a small quantized model and caching repeated report queries.
- **Integration delays:** three independently developed modules may have interface mismatches; mitigate with an agreed data-contract (schema/field names) fixed by Week 3, and a dedicated integration buffer (Weeks 12, 14).
- **Knowledge base quality:** a thin knowledge base leads to shallow RAG answers; mitigate by iteratively expanding factory notes based on gaps found during testing.

13 Summary

This project proposes MQIS, a deployable software platform that unifies real-time visual defect detection, structured event logging, live analytics, and RAG-based natural-language reporting for a manufacturing quality-control setting. It meets the course criteria by emphasizing incremental, testable delivery across

three clearly divided engineering roles, using measurable evaluation criteria (particularly mean time-to-log and detection consistency), and remaining focused on system integration and correctness rather than algorithmic novelty.